

МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»
Кафедра 806 «Вычислительная математика и программирование»

**Лабораторная работа № 2 по курсу
«Операционные системы и архитектура компьютера»**

Выполнил: *Д. А. Ефимова*

Группа: М8О-204БВ-25

Преподаватель: Н. А. Гапонов

Дата: *08.10.2026*

Оценка: _____

Москва, 2026

Оглавление

Оглавление.....	2
Условие	3
Описание программы	4
Структура файлов	4
Основные типы и функции	4
Используемые системные вызовы	4
Ключевые фрагменты кода	4
Результаты	7
Мои решения:	7
Тесты:	7
Результаты тестов:	8
Демонстрация работы strace	9
Выводы.....	10
Приложение А. Схема взаимодействия	11
Приложение Б. Исходный код программы	12
Приложение В. Полный вывод strace	15

Условие

Цель работы: приобретение практических навыков управления процессами в ОС Linux и обеспечения обмена данными между процессами через неименованные каналы (pipe).

Задание: составить и отладить программу на C++, осуществляющую работу с процессами и взаимодействие между ними. Родительский процесс создаёт дочерний, передаёт ему через pipe данные из файла, дочерний обрабатывает и возвращает результат через pipe. Обработать системные ошибки.

Вариант: №9 и задание варианта: в файле записаны команды вида «число число<newline>». Дочерний процесс производит деление первого числа команды на последующие, результат выводит в стандартный поток вывода. При делении на 0 дочерний и родительский процессы завершают работу. Числа типа float. Количество чисел произвольное.

Метод решения:

1) Родительский процесс (parent):

- Читает имя файла с консоли.
- Открывает файл через open.
- Создает pipe через pipe.
- Создает дочерний процесс через fork.
- В ребёнке: dup2 подменяет stdin на файл, stdout — на pipe; затем exec1 запускает child.
- В родителе: закрывает ненужные концы, читает из pipe, выводит результат, ждёт ребёнка через waitpid.

2) Дочерний процесс (child):

- Читает строки из stdin (это файл)
- Парсит числа через istream.
- Делит первое на последующие.
- Пишет результат в stdout (это pipe).
- При делении на 0 — exit(2); при плохом вводе — exit(3).

Смотреть приложение А – рисунок взаимодействия

Описание программы

Структура файлов

Таблица 1 – Файлы проекта

Файл	Назначение
src/parent.c	Родительский процесс: открывает файл, создаёт pipe и fork, читает результат
src/child.c	Дочерний процесс: читает stdin, делит числа, пишет в stdout
CMakeLists.txt	Сборка обеих программ

Основные типы и функции

std::istream — для парсинга строки.

std::getline — чтение строк из stdin.

std::cout/std::cerr — вывод.

std::strerror(errno) — обработка ошибок.

Используемые системные вызовы

Таблица 2 – Используемые системные вызовы

Системный вызов	Назначение в программе
int open(const char*, int)	Открытие файла с командами
int pipe(int fd[2])	Создание канала
pid_t fork(void)	Создание дочернего процесса
int dup2(int oldfd, int newfd)	Перенаправление std::cin/std::cout дочернего процесса на файл/канал
int close(int)	Закрытие дескрипторов
int execl(const char*, ..., NULL)	Запуск child
ssize_t read(int, void*, size_t)	Чтение из pipe
pid_t waitpid(pid_t, int*, int)	Ожидание завершения ребёнка
void _exit(int)	Аварийное завершение после неудачного execl
void exit(int)	Завершение дочернего процесса с нужным кодом

Ключевые фрагменты кода

Листинг 1 – Создание канала и проверка на ошибку (src/parent.c)

```
int pipefd[2];
if (pipe(pipefd) == -1) {
    std::cerr << "Error: pipe failed: " << std::strerror(errno) << '\n';
    return 1;
}
```

```

    }
    /*
    pipe(...) - возвращает 0 при успехе, -1 при ошибке
    pipefd[0] - чтение канала, pipefd[1] - запись в канал
    errno - числовой код причины ошибки
    std::strerror(errno) - превращает в понятную ошибку
    */

```

Листинг 2 – Создание дочернего процесса fork() (src/parent.c)

```

pid_t pid = fork();
if (pid == -1) { /*при ошибке возвращает -1*/
    std::cerr << "Error: fork failed: " << std::strerror(errno) << '\n';
    return 1;
}
else if (pid == 0) { /*если мы в дочернем процессе, то pid равен 0*/
    ...
}
else { /*иначе, если мы в родительском процессе, то pid равен id дочернего процесса*/
    ...
}

```

Листинг 3 – В дочернем процессе: перенаправляем std::cin/cout (src/parent.c)

```

/* Каждый процесс имеет таблицу открытых файловых дескрипторов.
По соглашению:
    0 – стандартный ввод (stdin, обычно std::cin)
    1 – стандартный вывод (stdout, обычно std::cout)
    2 – стандартный вывод ошибок (stderr, обычно std::cerr)
Далее дескрипторы выдаются по порядку при open(), pipe() и т.п.

dup2(oldfd, newfd) делает newfd копией oldfd:
после вызова оба дескриптора ссылаются на одно и то же открытое файловое описание, а
прежний newfd (если был открыт) закрывается. */

dup2(fd, 0); /* теперь дескриптор 0 читает из файла fd,
               т.е. std::cin читает не с клавиатуры, а из файла */
dup2(pipefd[1], 1); /* теперь дескриптор 1 пишет в pipefd[1],
                     т.е. std::cout пишет в канал, а не на экран */

close(pipefd[0]); /*читающий конец канала child process не нужен: мы в него не читаем */
close(fd); /* исходный дескриптор файла не нужен: дескриптор 0 уже его дублирует */
close(pipefd[1]); /* пишущий конец канала не нужен: дескриптор 1 уже его дублирует */

/*close(pipefd[0]) важен: если его не закрыть, читатель канала может не получить EOF*/

```

Листинг 4 – Запуск child (src/parent.c)

```

execl("./build/child", "child", NULL);
/*После успешного выполнения execl, дальше дочерний процесс в этом файле не пойдет
После неудачного execl ты находишься в дочернем процессе после fork()*/

std::cerr << "Error: execl failed: " << std::strerror(errno) << '\n';
_exit(127); //системный вызов, который не делает flush и не вызывает atexit
//код возврата 127 означает «команда не найдена / не удалось выполнить».

```

Листинг 5 – Чтение из pipe и ожидание (src/parent.c)

```

char buf[4096];

```

```
ssize_t n;
while ((n = read(pipefd[0], buf, sizeof(buf))) > 0) {
    std::cout.write(buf, n);
}
close(pipefd[0]);
int status;
waitpid(pid, &status, 0);
/*wait() ждёт любого ребёнка. waitpid позволяет указать конкретного и добавить флаги.*/
```

Результаты

Мои решения:

Я использую `_exit(127)` после `execl("./build/child", "child", NULL)` (см. Листинг 4), т.к.:

`return` : возврат из `main` → вызывает `exit()`

`exit(code)` : flush всех буферов, вызывает `atexit/on_exit`-обработчики, закрывает потоки
=> может дважды сбросить унаследованные от родителя буферы → дублирование вывода; может выполнить чужие обработчики.

`_exit(code)` : немедленно завершает процесс без flush, без `atexit`, без деструкторов

Тогда можно вынести две главные причины:

- 1) До `fork()` родитель мог что-то записать в `std::cout`, но не сбросить (буфер ещё не полон). При `fork()` буфер копируется в дочерний процесс. Если ребёнок вызовет `exit()` → буфер сбросится → на экран выведется та же порция дважды (родитель + ребёнок). `_exit()` этого не делает — буфер просто отбрасывается.
- 2) `exit()` дёргает `*atexit`-обработчики, деструкторы статических объектов, закрывает `FILE*`. После `fork()` это опасно: обработчики писались для родителя, а тут их запустит ребёнок — двойное закрытие, гонки, мусор.

`*atexit(func)` регистрирует функцию `func`, которая автоматически вызовется при нормальном завершении программы

Важный момент: У меня перед `_exit(127)` строка с выводом `std::cerr`. `std::cerr` по умолчанию `unbuffered (unitbuf)`, поэтому текст долетает до `stderr` сразу, без flush. То есть даже с `_exit` сообщение об ошибке успеет выйти.

Но если написать `std::cout` (буферизованный) и затем `_exit` — вывод потерялся бы. Поэтому здесь именно `cerr`.

Также у меня в коде есть `waitpid(pid, &status, 0)` (см. Листинг 5). Он нужен, чтобы:

- 1) Без этого родитель может завершиться раньше ребёнка
- 2) `waitpid` позволяет узнать как и с каким кодом завершился ребёнок
- 3) Завершившийся ребёнок не исчезает сразу. Он остаётся в таблице процессов как зомби (Z), пока родитель не вызовет `wait/waitpid`. Для этой лабораторной это не критично, но если бы родитель в цикле создавал много детей и не ждал - утечка ресурсов ядра => исчерпание таблицы процессов.

Тесты:

Тестовые данные без ошибок (`./tests/goodTest.txt`)

```
100 2 25
0
999 9 3
32 50
2 1 1 1 1
200 2000 10 10
1 2
```

Тестовые данные с ошибкой деление на 0 (./tests/byzeroTest.txt)

```
0 2 3 4
1 1 1 2

3 2

4 0 0 0
```

Тестовые данные с плохим вводом(./tests/badTest.txt)

```
100 2 25
999 9 3
akkjl
2 1 1 1 1
200 2000 10 10
1 2
```

Результаты тестов:

Вывод программы. На вход подаются уже созданные тестовые данные(они выше)

```
#Первый тест: ./tests/goodTest.txt
#Вывод:
2
0
37
0.64
2
0.001
0.5

#Первый тест: ./tests/byzeroTest.txt
#Вывод:
0
0.5
1.5
Error: деление на 0 в строке 6
child exited with code 2

#Первый тест: ./tests/badTest.txt
#Вывод:
2
37
Error: неправильный ввод в строке 3
child exited with code 3
```

Демонстрация работы strace

Программа запущена командой `strace -f ./build/parent`; Флаг `-f` позволяет отслеживать системные вызовы как родительского (PID 50236), так и дочернего (PID 50282) процессов. Полный вывод приведён в приложении Б.

```
pipe2([4, 5], 0) = 0
clone(child_stack=NULL, flags=CLONE_CHILD_CLEARTID|CLONE_CHILD_SETTID|SIGCHLD, ...) =
50282
```

Создан неименованный канал: дескриптор 4 — для чтения, 5 — для записи. Через `clone` (реализация `fork` в `glibc`) создан дочерний процесс с PID 50282.

```
[pid 50282] dup2(3, 0) = 0
[pid 50282] dup2(5, 1) = 1
[pid 50282] close(4) = 0
[pid 50282] close(3) = 0
[pid 50282] close(5) = 0
[pid 50282] execve("./build/child", ["child"], ...) = 0
```

В дочернем процессе `stdin` перенаправлен на файл (`dup2(3, 0)`), `stdout` — на канал (`dup2(5, 1)`). Лишние дескрипторы закрыты. Через `execve` процесс заменил образ на программу `child`, сохранив PID 50282.

```
[pid 50236] close(5) = 0
[pid 50236] read(4, <unfinished ...>
[pid 50282] write(1, "2\n", 2) = 2
[pid 50236] <... read resumed>"2\n", 4096) = 2
[pid 50236] write(1, "2\n", 2) = 2
```

Родитель закрыл писатель канала и начал чтение. Дочерний процесс записал в канал результат (2\n), родитель прочитал его и вывел в свой `stdout`.

```
[pid 50282] exit_group(0) = ?
[pid 50236] read(4, "", 4096) = 0
--- SIGCHLD {si_pid=50282, si_status=0, ...} ---
wait4(50282, [{WIFEXITED(s) && WEXITSTATUS(s) == 0}], 0, NULL) = 50282
exit_group(0) = ?
```

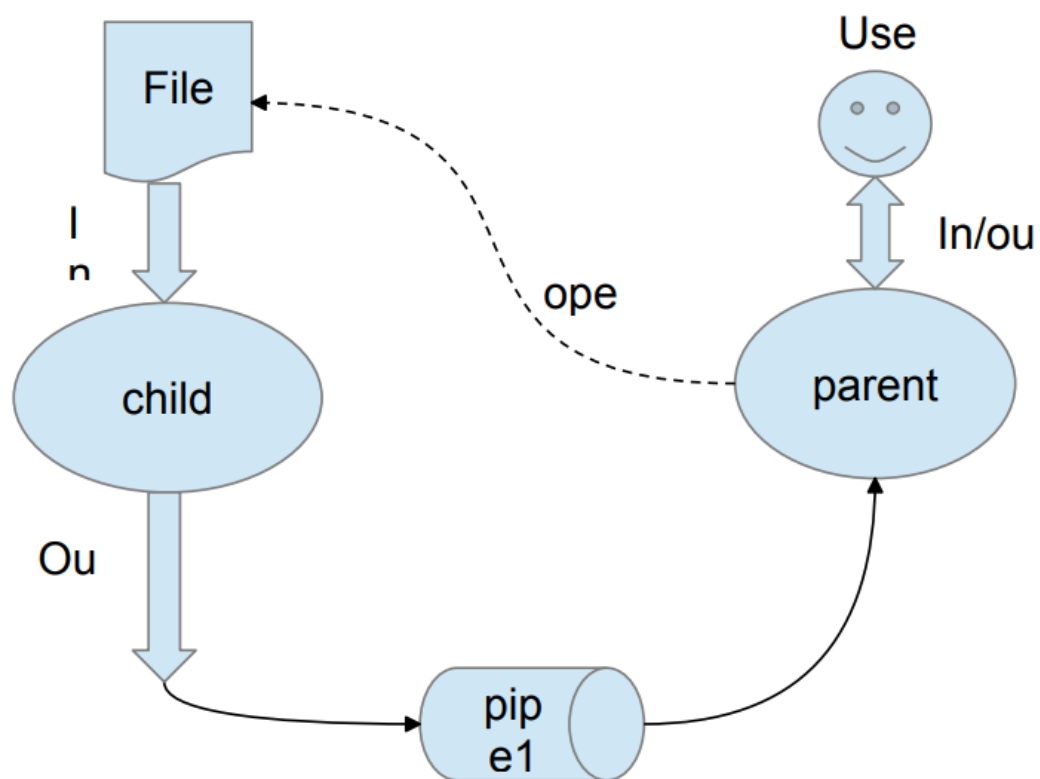
Дочерний процесс завершился с кодом 0. Родитель получил сигнал `SIGCHLD`, дочитал канал до EOF (`read` вернул 0) и через `wait4` дождался завершения ребёнка. Статус подтверждает нормальное завершение.

Выводы

В ходе работы реализовано взаимодействие родительского и дочернего процессов через неименованный канал (pipe). Родитель открывает файл и через dup2 подменяет stdin ребёнка, а stdout ребёнка направляет в pipe. Ребёнок читает команды, делит числа и возвращает результат. Обработаны ошибки: деление на ноль (exit(2)), сбой системных вызовов (errno/streerror). Подтверждена корректность через strace и тесты. Трудность: нестабильный порядок вывода между stdout и stderr из-за разных потоков — решена добавлением flush.

Приложение А. Схема взаимодействия

Фото А.1 – схема взаимодействия 9 вариант



Приложение Б. Исходный код программы

Листинг Б.1 – Файл src/parent.cpp

```
#include <iostream>
#include <string>
#include <fcntl.h>      // open
#include <unistd.h>
#include <cstdlib>      // exit
#include <sys/wait.h>   // waitpid, WIFEXITED, WEXITSTATUS
#include <cerrno>
#include <cstring>      // std::strerror

int main() {
    std::string path;
    std::cout << "Введите имя файла: ";
    std::getline(std::cin, path);

    int fd = open(path.c_str(), O_RDONLY);
    if (fd == -1) {
        std::cerr << "Error: open failed: " << std::strerror(errno) << '\n';
        return 1;
    }

    int pipefd[2];
    if (pipe(pipefd) == -1) {
        std::cerr << "Error: pipe failed: " << std::strerror(errno) << '\n';
        return 1;
    }

    pid_t pid = fork();
    if (pid == -1) {
        std::cerr << "Error: fork failed: " << std::strerror(errno) << '\n';
        return 1;
    }
    else if (pid == 0) {
        dup2(fd, 0);
        dup2(pipefd[1], 1);

        close(pipefd[0]);
        close(fd);
        close(pipefd[1]);

        execl("./build/child", "child", NULL);

        std::cerr << "Error: execl failed: " << std::strerror(errno) << '\n';
        _exit(127); //системный вызов, который не делает flush и не вызывает atexit
    }
    else {
        close(pipefd[1]);
        close(fd);

        char buf[4096];
        ssize_t n;
        while ((n = read(pipefd[0], buf, sizeof(buf))) > 0) {
            std::cout.write(buf, n);
            std::cout << std::flush;
        }
        if (n == -1) {
```

```

        std::cerr << "read failed: " << std::strerror(errno) << '\n';
    }

    close(pipefd[0]);

    int status;
    if (waitpid(pid, &status, 0) == -1) {
        std::cerr << "waitpid failed: " << std::strerror(errno) << '\n';
        return 1;
    }
    if (WIFEXITED(status)) {
        int code = WEXITSTATUS(status);
        if (code != 0) {
            std::cerr << "child exited with code " << code << '\n';
            return 1;
        }
    } else if (WIFSIGNALED(status)) {
        std::cerr << "child killed by signal " << WTERMSIG(status) << '\n';
        return 1;
    }

    return 0;
}
}

```

Листинг Б.2 – Файл child.cpp

```

#include <iostream>
#include <sstream>
#include <string>
#include <sys/wait.h>

int main() {
    //число число число<endl>
    // Дочерний процесс производит деление первого числа команда, на последующие числа в
    команде

    std::string line;
    int line_number = 0;
    while (std::getline(std::cin, line)) {
        line_number++;

        std::istringstream iss(line);
        double res = 0, x = 0;
        iss >> res;
        if (iss.fail() && iss.eof()) continue;

        while (iss >> x) {
            if (x == 0) {
                std::cout << "Error: деление на 0 в строке " << line_number << '\n';
                return 2;
            }
            res /= x;
        }
        if (iss.fail() && !iss.eof()) {
            std::cout << "Error: неправильный ввод в строке " << line_number << '\n';
            return 3;
        }

        std::cout << res << std::endl; //std::endl делает flash автоматически
    }
}

```

```
    }  
    return 0;  
}
```

Листинг Б.3 – Файл CMakefile.txt

```
cmake_minimum_required(VERSION 3.10)  
project(lab2)  
  
set(CMAKE_CXX_STANDARD 17)  
  
add_compile_options(-Wall -Wextra)  
  
add_executable(parent src/parent.cpp)  
add_executable(child src/child.cpp)
```

Приложение В. Полный вывод strace

Листинг В.1 – Вывод strace -f ./build/parent (для тестовых данных: ./tests/goodTest.txt)

```
execve("./build/parent", ["/build/parent"], 0x7ffea25f03a8 /* 41 vars */) = 0
brk(NULL)                               = 0x592da1bc8000
mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) =
0x7a19a70de000
access("/etc/ld.so.preload", R_OK)      = -1 ENOENT (No such file or directory)
openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
fstat(3, {st_mode=S_IFREG|0644, st_size=21411, ...}) = 0
mmap(NULL, 21411, PROT_READ, MAP_PRIVATE, 3, 0) = 0x7a19a70d8000
close(3)                                 = 0
openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libstdc++.so.6", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\0\0\0\0\0\0\0\0\0"..., 832) = 832
fstat(3, {st_mode=S_IFREG|0644, st_size=2592224, ...}) = 0
mmap(NULL, 2609472, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) = 0x7a19a6e00000
mmap(0x7a19a6e9d000, 1343488, PROT_READ|PROT_EXEC, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE,
3, 0x9d000) = 0x7a19a6e9d000
mmap(0x7a19a6fe5000, 552960, PROT_READ, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3,
0x1e5000) = 0x7a19a6fe5000
mmap(0x7a19a706c000, 57344, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE,
3, 0x26b000) = 0x7a19a706c000
mmap(0x7a19a707a000, 12608, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS, -
1, 0) = 0x7a19a707a000
close(3)                                 = 0
openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libgcc_s.so.1", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\0\0\0\0\0\0\0\0\0"..., 832) = 832
fstat(3, {st_mode=S_IFREG|0644, st_size=183024, ...}) = 0
mmap(NULL, 185256, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) = 0x7a19a70aa000
mmap(0x7a19a70ae000, 147456, PROT_READ|PROT_EXEC, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE,
3, 0x4000) = 0x7a19a70ae000
mmap(0x7a19a70d2000, 16384, PROT_READ, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x28000)
= 0x7a19a70d2000
mmap(0x7a19a70d6000, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3,
0x2b000) = 0x7a19a70d6000
close(3)                                 = 0
openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\0\220\243\2\0\0\0\0\0"..., 832) =
832
pread64(3, "\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0"..., 784, 64)
= 784
fstat(3, {st_mode=S_IFREG|0755, st_size=2129424, ...}) = 0
pread64(3, "\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0"..., 784, 64)
= 784
mmap(NULL, 2174352, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) = 0x7a19a6a00000
mmap(0x7a19a6a28000, 1609728, PROT_READ|PROT_EXEC, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE,
3, 0x28000) = 0x7a19a6a28000
mmap(0x7a19a6bb1000, 323584, PROT_READ, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3,
0x1b1000) = 0x7a19a6bb1000
mmap(0x7a19a6c00000, 24576, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE,
3, 0x1ff000) = 0x7a19a6c00000
mmap(0x7a19a6c06000, 52624, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS, -
1, 0) = 0x7a19a6c06000
close(3)                                 = 0
openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libm.so.6", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\0\0\0\0\0\0\0\0\0"..., 832) = 832
fstat(3, {st_mode=S_IFREG|0644, st_size=952616, ...}) = 0
mmap(NULL, 950296, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) = 0x7a19a6d17000
```

```

mmap(0x7a19a6d27000, 520192, PROT_READ|PROT_EXEC, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE,
3, 0x10000) = 0x7a19a6d27000
mmap(0x7a19a6da6000, 360448, PROT_READ, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x8f000)
= 0x7a19a6da6000
mmap(0x7a19a6dfe000, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3,
0xe7000) = 0x7a19a6dfe000
close(3) = 0
mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) =
0x7a19a70a8000
mmap(NULL, 12288, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) =
0x7a19a70a5000
arch_prctl(ARCH_SET_FS, 0x7a19a70a5740) = 0
set_tid_address(0x7a19a70a5a10) = 50236
set_robust_list(0x7a19a70a5a20, 24) = 0
rseq(0x7a19a70a6060, 0x20, 0, 0x53053053) = 0
mprotect(0x7a19a6c00000, 16384, PROT_READ) = 0
mprotect(0x7a19a6dfe000, 4096, PROT_READ) = 0
mprotect(0x7a19a70d6000, 4096, PROT_READ) = 0
mprotect(0x7a19a706c000, 45056, PROT_READ) = 0
mprotect(0x592d82e74000, 4096, PROT_READ) = 0
mprotect(0x7a19a711e000, 8192, PROT_READ) = 0
prlimit64(0, RLIMIT_STACK, NULL, {rlim_cur=8192*1024, rlim_max=RLIM64_INFINITY}) = 0
munmap(0x7a19a70d8000, 21411) = 0
futex(0x7a19a707a7bc, FUTEX_WAKE_PRIVATE, 2147483647) = 0
getrandom("\x52\x03\xf3\xdf\xaa\x7e\x49\x64", 8, GRND_NONBLOCK) = 8
brk(NULL) = 0x592da1bc8000
brk(0x592da1be9000) = 0x592da1be9000
fstat(1, {st_mode=S_IFCHR|0620, st_rdev=makedev(0x88, 0xb), ...}) = 0
write(1, "\320\222\320\262\320\265\320\264\320\270\321\202\320\265
\320\270\320\274\321\217 \321\204\320\260\320\271\320\273\320\260"... , 34Введите имя
файла: ) = 34
fstat(0, {st_mode=S_IFCHR|0620, st_rdev=makedev(0x88, 0xb), ...}) = 0
read(0, ./tests/goodTest.txt
"./tests/goodTest.txt\n", 1024) = 21
openat(AT_FDCWD, "./tests/goodTest.txt", O_RDONLY) = 3
pipe2([4, 5], 0) = 0
clone(child_stack=NULL, flags=CLONE_CHILD_CLEARTID|CLONE_CHILD_SETTID|SIGCHLDstrace:
Process 50282 attached
, child_tidptr=0x7a19a70a5a10) = 50282
[pid 50282] set_robust_list(0x7a19a70a5a20, 24 <unfinished ...>
[pid 50236] close(5 <unfinished ...>
[pid 50282] <... set_robust_list resumed>) = 0
[pid 50236] <... close resumed>) = 0
[pid 50282] dup2(3, 0 <unfinished ...>
[pid 50236] close(3 <unfinished ...>
[pid 50282] <... dup2 resumed>) = 0
[pid 50236] <... close resumed>) = 0
[pid 50282] dup2(5, 1 <unfinished ...>
[pid 50236] read(4, <unfinished ...>
[pid 50282] <... dup2 resumed>) = 1
[pid 50282] close(4) = 0
[pid 50282] close(3) = 0
[pid 50282] close(5) = 0
[pid 50282] execve("./build/child", ["child"], 0x7ffca1f648b8 /* 41 vars */) = 0
[pid 50282] brk(NULL) = 0x60af09a1b000
[pid 50282] mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) =
0x7a4a4fdf7e000
[pid 50282] access("/etc/ld.so.preload", R_OK) = -1 ENOENT (No such file or directory)
[pid 50282] openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
[pid 50282] fstat(3, {st_mode=S_IFREG|0644, st_size=21411, ...}) = 0

```

```

[pid 50282] mmap(NULL, 21411, PROT_READ, MAP_PRIVATE, 3, 0) = 0x74a4fdf78000
[pid 50282] close(3) = 0
[pid 50282] openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libstdc++.so.6", O_RDONLY|O_CLOEXEC)
= 3
[pid 50282] read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\0\0\0\0\0"...,
832) = 832
[pid 50282] fstat(3, {st_mode=S_IFREG|0644, st_size=2592224, ...}) = 0
[pid 50282] mmap(NULL, 2609472, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) =
0x74a4fdc00000
[pid 50282] mmap(0x74a4fdc9d000, 1343488, PROT_READ|PROT_EXEC,
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x9d000) = 0x74a4fdc9d000
[pid 50282] mmap(0x74a4fdde5000, 552960, PROT_READ, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE,
3, 0x1e5000) = 0x74a4fdde5000
[pid 50282] mmap(0x74a4fde6c000, 57344, PROT_READ|PROT_WRITE,
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x26b000) = 0x74a4fde6c000
[pid 50282] mmap(0x74a4fde7a000, 12608, PROT_READ|PROT_WRITE,
MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS, -1, 0) = 0x74a4fde7a000
[pid 50282] close(3) = 0
[pid 50282] openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libgcc_s.so.1", O_RDONLY|O_CLOEXEC)
= 3
[pid 50282] read(3, "\177ELF\2\1\1\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\0\0\0\0\0"...,
832) = 832
[pid 50282] fstat(3, {st_mode=S_IFREG|0644, st_size=183024, ...}) = 0
[pid 50282] mmap(NULL, 185256, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) =
0x74a4fdf4a000
[pid 50282] mmap(0x74a4fdf4e000, 147456, PROT_READ|PROT_EXEC,
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x4000) = 0x74a4fdf4e000
[pid 50282] mmap(0x74a4fdf72000, 16384, PROT_READ, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE,
3, 0x28000) = 0x74a4fdf72000
[pid 50282] mmap(0x74a4fdf76000, 8192, PROT_READ|PROT_WRITE,
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x2b000) = 0x74a4fdf76000
[pid 50282] close(3) = 0
[pid 50282] openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC) = 3
[pid 50282] read(3,
"\177ELF\2\1\1\3\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\220\243\2\0\0\0\0"..., 832) = 832
[pid 50282] pread64(3,
"\6\0\0\0\4\0\0\0@ \0\0\0\0\0\0\0\0@ \0\0\0\0\0\0\0\0"..., 784, 64) = 784
[pid 50282] fstat(3, {st_mode=S_IFREG|0755, st_size=2129424, ...}) = 0
[pid 50282] pread64(3,
"\6\0\0\0\4\0\0\0@ \0\0\0\0\0\0\0\0@ \0\0\0\0\0\0\0\0"..., 784, 64) = 784
[pid 50282] mmap(NULL, 2174352, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) =
0x74a4fd800000
[pid 50282] mmap(0x74a4fd828000, 1609728, PROT_READ|PROT_EXEC,
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x28000) = 0x74a4fd828000
[pid 50282] mmap(0x74a4fd9b1000, 323584, PROT_READ, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE,
3, 0x1b1000) = 0x74a4fd9b1000
[pid 50282] mmap(0x74a4fda00000, 24576, PROT_READ|PROT_WRITE,
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x1ff000) = 0x74a4fda00000
[pid 50282] mmap(0x74a4fda06000, 52624, PROT_READ|PROT_WRITE,
MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS, -1, 0) = 0x74a4fda06000
[pid 50282] close(3) = 0
[pid 50282] openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libm.so.6", O_RDONLY|O_CLOEXEC) = 3
[pid 50282] read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\0\0\0\0\0"...,
832) = 832
[pid 50282] fstat(3, {st_mode=S_IFREG|0644, st_size=952616, ...}) = 0
[pid 50282] mmap(NULL, 950296, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) =
0x74a4fdb17000
[pid 50282] mmap(0x74a4fdb27000, 520192, PROT_READ|PROT_EXEC,
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x10000) = 0x74a4fdb27000

```

```

[pid 50282] mmap(0x74a4fdb6000, 360448, PROT_READ, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE,
3, 0x8f000) = 0x74a4fdb6000
[pid 50282] mmap(0x74a4fdbfe000, 8192, PROT_READ|PROT_WRITE,
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0xe7000) = 0x74a4fdbfe000
[pid 50282] close(3) = 0
[pid 50282] mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) =
0x74a4fdf48000
[pid 50282] mmap(NULL, 12288, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) =
0x74a4fdf45000
[pid 50282] arch_prctl(ARCH_SET_FS, 0x74a4fdf45740) = 0
[pid 50282] set_tid_address(0x74a4fdf45a10) = 50282
[pid 50282] set_robust_list(0x74a4fdf45a20, 24) = 0
[pid 50282] rseq(0x74a4fdf46000, 0x20, 0, 0x53053053) = 0
[pid 50282] mprotect(0x74a4fda00000, 16384, PROT_READ) = 0
[pid 50282] mprotect(0x74a4fdbfe000, 4096, PROT_READ) = 0
[pid 50282] mprotect(0x74a4fdf76000, 4096, PROT_READ) = 0
[pid 50282] mprotect(0x74a4fde6c000, 45056, PROT_READ) = 0
[pid 50282] mprotect(0x60aedf6be000, 4096, PROT_READ) = 0
[pid 50282] mprotect(0x74a4fdfbe000, 8192, PROT_READ) = 0
[pid 50282] prlimit64(0, RLIMIT_STACK, NULL, {rlim_cur=8192*1024,
rlim_max=RLIM64_INFINITY}) = 0
[pid 50282] munmap(0x74a4fdf78000, 21411) = 0
[pid 50282] futex(0x74a4fde7a7bc, FUTEX_WAKE_PRIVATE, 2147483647) = 0
[pid 50282] getrandom("\x38\x03\x19\xc8\x60\x66\xd2\xc1", 8, GRND_NONBLOCK) = 8
[pid 50282] brk(NULL) = 0x60af09a1b000
[pid 50282] brk(0x60af09a3c000) = 0x60af09a3c000
[pid 50282] fstat(0, {st_mode=S_IFREG|0644, st_size=63, ...}) = 0
[pid 50282] read(0, "100 2 25\n\n0\n999 9 3 \n\n32 50\n2 1 "..., 4096) = 63
[pid 50282] fstat(1, {st_mode=S_IFIFO|0600, st_size=0, ...}) = 0
[pid 50282] write(1, "2\n", 2 <unfinished ...>
[pid 50236] <... read resumed>"2\n", 4096) = 2
[pid 50282] <... write resumed> = 2
[pid 50236] write(1, "2\n", 2 <unfinished ...>
[pid 50282] write(1, "0\n", 22
<unfinished ...>
[pid 50236] <... write resumed> = 2
[pid 50282] <... write resumed> = 2
[pid 50236] read(4, "0\n", 4096) = 2
[pid 50282] write(1, "37\n", 3 <unfinished ...>
[pid 50236] write(1, "0\n", 2 <unfinished ...>
0
[pid 50282] <... write resumed> = 3
[pid 50236] <... write resumed> = 2
[pid 50282] write(1, "0.64\n", 5 <unfinished ...>
[pid 50236] read(4, <unfinished ...>
[pid 50282] <... write resumed> = 5
[pid 50236] <... read resumed>"37\n0.64\n", 4096) = 8
[pid 50236] write(1, "37\n0.64\n", 8 <unfinished ...>
[pid 50282] write(1, "2\n", 237
0.64
<unfinished ...>
[pid 50236] <... write resumed> = 8
[pid 50282] <... write resumed> = 2
[pid 50236] read(4, <unfinished ...>
[pid 50282] write(1, "0.001\n", 6 <unfinished ...>
[pid 50236] <... read resumed>"2\n", 4096) = 2
[pid 50282] <... write resumed> = 6
[pid 50236] write(1, "2\n", 22
<unfinished ...>
[pid 50282] write(1, "0.5\n", 4 <unfinished ...>

```

```

[pid 50236] <... write resumed>          = 2
[pid 50236] read(4, <unfinished ...>
[pid 50282] <... write resumed>          = 4
[pid 50236] <... read resumed>"0.001\n0.5\n", 4096) = 10
[pid 50282] read(0, <unfinished ...>
[pid 50236] write(1, "0.001\n0.5\n", 10 <unfinished ...>
[pid 50282] <... read resumed>"", 4096) = 0
0.001
0.5
[pid 50236] <... write resumed>          = 10
[pid 50236] read(4, <unfinished ...>
[pid 50282] exit_group(0)                = ?
[pid 50236] <... read resumed>"", 4096) = 0
[pid 50282] +++ exited with 0 +++
--- SIGCHLD {si_signo=SIGCHLD, si_code=CLD_EXITED, si_pid=50282, si_uid=1000,
si_status=0, si_utime=0, si_stime=1 /* 0.01 s */} ---
close(4)                                = 0
wait4(50282, [{WIFEXITED(s) && WEXITSTATUS(s) == 0}], 0, NULL) = 50282
exit_group(0)                            = ?
+++ exited with 0 +++

```